DEVOPS WINTER ARC

DAY 21 / 90

Kubernetes Security, RBAC & Troubleshooting

Level	Intermediate	**Time**	3–4 hours
Focus	RBAC · ServiceAccounts · securityContext · Pod Security · Debugging	**Cluster**	Existing kind cluster

Today’s mission

A teammate needs read-only access to one namespace. The platform team wants hardened Pods only. Three broken Deployments then need fixing. Work like an on-call DevOps engineer: gather evidence first, then make the smallest safe change.

Scope rule: Do not repeat or redo topics/tasks from Days 15–20. Today covers only RBAC, ServiceAccounts, securityContext, Pod Security Standards, and debugging the three broken Deployments.

Learning outcomes

• Create a dedicated ServiceAccount and grant only required permissions.

• Verify allowed and denied actions with kubectl auth can-i.

• Run a Pod as non-root with a hardened securityContext.

• Enforce the restricted Pod Security profile on a namespace.

• Diagnose and fix three broken Deployments using Events and logs.

Part 1 — Key concepts (30%)

RBAC: who can do what, where

• **ServiceAccount:** identity used by Pods or tools.

• **Role:** permissions in one namespace. **ClusterRole:** cluster-scoped or reusable permissions.

• **RoleBinding:** connects a Role to a subject. RBAC is deny-by-default; grant only necessary verbs/resources.

• **Least privilege:** a read-only viewer should not be able to delete Pods.

Pod security

• **runAsNonRoot / runAsUser:** run the process as a non-root user.

• **readOnlyRootFilesystem:** make the container root filesystem read-only.

• **allowPrivilegeEscalation: false:** prevent privilege escalation.

• **capabilities.drop: [ALL]:** remove unneeded Linux capabilities.

• **seccompProfile: RuntimeDefault:** use the runtime’s default seccomp profile.

• **Pod Security Standards:** privileged, baseline, restricted; restricted is the strongest profile.

Repeatable debug routine

get → **describe** → **logs** → **events** → **exec**. Start with the Pod status and read Events before guessing. Use **logs --previous** when a container has crashed.

Symptom	Likely cause / first check
Pending	Scheduling/resource issue or unbound PVC. Run describe pod and inspect Events.
ImagePullBackOff	Wrong image/tag, registry access, or image not loaded into kind. Check Events.
CrashLoopBackOff	App exits, command/config is wrong, or process is killed. Check logs --previous and describe.
Running, not working	Check logs and object details; verify the actual failure before changing anything.

Part 2 — Hands-on lab (70%)

A. Create a namespace and read-only identity

Use your existing kind cluster. Do not recreate it.

```
kubectl create namespace secure
kubectl create serviceaccount viewer -n secure
kubectl create namespace lab
```

Create **rbac.yaml**:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: secure
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: viewer-pods
  namespace: secure
subjects:
  - kind: ServiceAccount
    name: viewer
    namespace: secure
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f rbac.yaml
kubectl auth can-i get pods -n secure \
  --as=system:serviceaccount:secure:viewer
kubectl auth can-i delete pods -n secure \
  --as=system:serviceaccount:secure:viewer
```

Expected: get pods → yes; delete pods → no. If not, inspect the Role and RoleBinding.

B. Enforce Pod Security and harden a Pod

```
kubectl label namespace secure \
  pod-security.kubernetes.io/enforce=restricted
```

Create **secure-pod.yaml**:

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  namespace: secure
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: busybox:1.36
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
```

```
kubectl apply -f secure-pod.yaml
kubectl get pod secure-pod -n secure
kubectl describe pod secure-pod -n secure
kubectl run root-pod --image=nginx:1.27 -n secure
```

The root-based Pod should be rejected by the restricted policy. Keep the policy enabled; do not weaken it just to admit the test Pod.

C. Create three broken Deployments

```
kubectl create deploy broken1 -n lab --image=nginx:1.99.99
kubectl create deploy broken2 -n lab --image=busybox:1.36 \
  -- sh -c "exit 1"
kubectl create deploy broken3 -n lab --image=nginx:1.27
kubectl set resources deploy broken3 -n lab --requests=memory=64Gi
```

Inspect status, Events and previous logs. Record each cause before fixing it.

```
kubectl get deployments,pods -n lab
kubectl describe pod <pod-name> -n lab
kubectl logs <pod-name> -n lab --previous
kubectl get events -n lab --sort-by=.lastTimestamp
```

Deployment	Expected symptom	Cause	Fix direction
broken1	ImagePullBackOff	Invalid image tag	Set a valid nginx image tag.
broken2	CrashLoopBackOff	Command exits with code 1	Change command to a valid long-running command.
broken3	Pending	64Gi request cannot fit on kind node	Reduce memory request to a realistic value.

Example repair commands; choose a valid image tag for your environment:

```
kubectl set image deployment/broken1 -n lab nginx=nginx:1.27
kubectl set resources deployment/broken3 -n lab --requests=memory=64Mi
```

For broken2, update the container command to a long-running command. Verify all three Deployments and their Pods reach Running.

Part 3 — Verification & deliverables

Completion checklist

■ viewer can read Pods/logs but cannot delete Pods in secure.

■ Restricted policy rejects the root Pod; secure-pod is admitted and runs as non-root.

■ All three lab Deployments are repaired and their Pods reach Running.

■ You documented symptom, evidence, root cause and fix.

Debug notes table

Deployment	Symptom / evidence	Root cause	Fix / final status
broken1			
broken2			
broken3			

GitHub deliverable

Create **day21-security-debug/** containing rbac.yaml, secure-pod.yaml, and README.md with the RBAC yes/no checks, Pod Security rejection evidence, debug routine, and a table of the three apps (symptom, cause, fix).

• Screenshot: kubectl auth can-i shows yes for get and no for delete.

• Screenshot: restricted policy rejects root Pod.

• Screenshot: all three repaired Pods are Running.

Part 4 — Interview questions

Answer aloud in 20–30 seconds before reading the suggested answer.

1. What is RBAC?

Suggested answer: It controls who can perform which actions on which resources and within what scope.

2. Role vs ClusterRole?

Suggested answer: Role is namespace-scoped; ClusterRole can apply cluster-wide or be reused across namespaces.

3. What does a RoleBinding do?

Suggested answer: It binds a Role to a subject such as a user, group or ServiceAccount in a namespace.

4. How do you verify permissions?

Suggested answer: Use kubectl auth can-i with the verb, resource, namespace and optionally --as.

5. Why avoid running as root?

Suggested answer: Non-root execution reduces the impact of a compromised or faulty container.

6. What is securityContext?

Suggested answer: Pod/container security settings such as user ID, privilege escalation, filesystem access and capabilities.

7. What does readOnlyRootFilesystem do?

Suggested answer: It prevents writes to the container root filesystem; writable paths need an appropriate volume.

8. What are Pod Security Standards?

Suggested answer: Profiles named privileged, baseline and restricted.

9. Why was the root Pod rejected?

Suggested answer: The namespace enforces restricted and admission rejects Pods that violate its requirements.

10. A Pod is Pending. First step?

Suggested answer: Describe the Pod and read Events for scheduling constraints or insufficient resources.

11. What does ImagePullBackOff mean?

Suggested answer: Kubernetes cannot pull the image; check its name/tag, registry access and Events.

12. How do you debug CrashLoopBackOff?

Suggested answer: Check logs --previous and describe output, including exit code and Events.

13. Why use least privilege?

Suggested answer: It limits accidental or malicious actions to only what the identity needs.

14. What is the debug order?

Suggested answer: get → describe → logs → events → exec; check status and Events before changing things.

Optional cleanup

After saving your evidence, delete only the namespaces created for this lab. This removes their resources.

```
kubectl delete namespace secure lab
```

Do not delete your shared kind cluster if you still need it for other Winter Arc tasks.

Day 21 complete: least-privilege access verified, hardened Pod admitted, root Pod blocked, and three broken Deployments fixed and explained.